The previous program tries to return from the scope where p is defined. Because you can't return from the top-level scope, the program fails. You can avoid this kind of mistake if you avoid using explicit returns:

```
p = Proc.new { 10 }
double(p)   # => 20
```

Now on to the second important difference between procs and lambdas.

Procs, Lambdas, and Arity

The second difference between procs and lambdas concerns the way they check their arguments. For example, a particular proc or lambda might have an *arity* of two, meaning that it accepts two arguments:

```
p = Proc.new {|a, b| [a, b]}
p.arity # => 2
```

What happens if you call this callable object with three arguments, or one single argument? The long answer to this question is complicated and littered with special cases.[1] The short answer is that, in general, lambdas tend to be less tolerant than procs (and regular blocks) when it comes to arguments. Call a lambda with the wrong arity, and it fails with an ArgumentError. On the other hand, a proc fits the argument list to its own expectations:

```
p = Proc.new {|a, b| [a, b]}
p.call(1, 2, 3)   # => [1, 2]
p.call(1)         # => [1, nil]
```

If there are too many arguments, a proc drops the excess arguments. If there are too few arguments, it assigns nil to the missing arguments.

Procs vs. Lambdas: The Verdict

You now know the differences between procs and lambdas. But you're wondering which kind of Proc you should use in your own code.

Generally speaking, lambdas are more intuitive than procs because they're more similar to methods. They're pretty strict about arity, and they simply exit when you call return. For this reason, many Rubyists use lambdas as a first choice, unless they need the specific features of procs.

1. A program to explore those special cases, written by Paul Cantrell, is at http://innig.net/software/ruby/closures-in-ruby.rb.